

DATA CLEANING & REPORTING AUTOMATION

Project Documentation

A Python-based web application for automated data cleaning, validation, analysis, visualization, and report generation.

Project Type	Data Analytics / Data Cleaning / Reporting Automation
Primary Language	Python
Web Framework	Streamlit
Data Formats	CSV and Excel (XLSX)
Deployment Target	Streamlit Community Cloud

Table of Contents

1. Project Overview
2. Problem Statement
3. Objectives
4. Key Features
5. Technology Stack
6. System Architecture
7. Project Workflow
8. Project Structure
9. Module Descriptions
10. Data Cleaning Process
11. Data Validation
12. Data Analysis and KPIs
13. Automatic Visualization
14. Automatic Insights
15. Report Generation
16. User Interface and Animations
17. Input Data and Testing Dataset
18. Error Handling and Limitations
19. How to Run the Project Locally
20. GitHub and Streamlit Deployment
21. User Guide
22. Testing Checklist
23. Future Enhancements
24. Conclusion

1. Project Overview

Data Cleaning & Reporting Automation is a Python-based Streamlit application designed to reduce the manual effort involved in preparing datasets for analysis. The application accepts CSV or Excel files, inspects the data, performs automated cleaning, validates data quality, calculates general-purpose KPIs, generates visualizations and insights, and creates downloadable reports.

The project is designed as a reusable data-processing tool rather than a solution for one fixed dataset. It can work with structured tabular datasets containing numeric, categorical, and date-related columns.

2. Problem Statement

Raw datasets commonly contain duplicate records, inconsistent text formatting, empty rows or columns, missing values, mixed date formats, invalid dates, and numeric values stored as text. Cleaning these problems manually is repetitive, time-consuming, and prone to human error.

The project solves this problem by providing a single interface where a user can upload a dataset and automatically perform common preparation, quality checking, analysis, visualization, and reporting tasks.

3. Objectives

- Automate common data-cleaning operations.
- Identify basic data-quality problems before and after cleaning.
- Remove duplicate records and unnecessary empty rows or columns.
- Standardize text values by removing unwanted surrounding spaces.
- Detect likely date and numeric columns automatically.
- Handle missing numeric values using median-based imputation.
- Generate general-purpose KPIs and statistical summaries.
- Create automatic charts based on detected column types.
- Generate simple data-driven insights.
- Produce downloadable cleaned CSV and Excel reports.
- Provide a user-friendly web interface through Streamlit.
- Prepare the application for GitHub-based deployment and sharing.

4. Key Features

Feature	Description
CSV/XLSX Upload	Users can upload structured CSV or Excel files.
Data Quality Check	Shows row count, column count, duplicates and missing values.
Automated Cleaning	Cleans column names, empty records, duplicates, text, dates and numeric fields.
Before/After Comparison	Shows how much data was changed or removed by the cleaning process.
Cleaning Impact	Uses progress indicators to communicate cleaning results.
KPI Dashboard	Displays dataset-level metrics in dashboard cards.
Automatic Charts	Creates numeric, categorical, date-trend and correlation charts when appropriate.
Automatic Insights	Produces readable observations from numeric and categorical columns.
Cleaned Dataset Preview	Displays the processed data inside the web application.
CSV Export	Allows users to download the cleaned dataset.
Excel Report	Creates a multi-sheet automated report.
Animations	Uses loading/progress feedback to make long operations clearer.

5. Technology Stack

Technology	Purpose
Python	Core programming language.
Pandas	Data loading, cleaning, transformation and analysis.
NumPy	Numerical operations and supporting data processing.
Matplotlib	Chart generation.
Seaborn	Installed for statistical visualization support and future enhancement.
OpenPyXL	Excel workbook generation.
Streamlit	Interactive web application framework.
Git/GitHub	Version control and project hosting.
Streamlit Community Cloud	Planned cloud deployment and sharing.
VS Code	Development environment.

6. System Architecture

The application follows a modular architecture. The Streamlit interface in `app.py` coordinates the workflow, while separate modules handle cleaning, validation, analysis, visualization and report generation.

Raw CSV/Excel → Data Loading → Inspection → Cleaning → Validation → Analysis → Visualization/Insights → Report Generation → Download

7. Project Workflow

1. User opens the Streamlit application.
2. User uploads a CSV or XLSX dataset.
3. The application reads the file into a Pandas DataFrame.
4. Initial data-quality information is calculated.
5. The user starts the cleaning and analysis process.
6. Duplicate records, empty records, text formatting issues and detectable date/numeric inconsistencies are processed.
7. Missing numeric values are filled using median values.
8. The cleaned dataset is validated again.
9. Cleaning impact is displayed using before/after metrics and progress indicators.
10. KPIs, summaries, charts and automatic insights are generated.
11. The cleaned data can be downloaded as CSV.
12. A multi-sheet Excel report can be generated and downloaded.

8. Project Structure

```
Data-Cleaning-Reporting-Automation/  
├─ app.py  
├─ requirements.txt  
├─ README.md  
├─ .gitignore  
├─ data/  
├─ modules/
```

```

|   |— cleaning.py
|   |— analysis.py
|   |— visualization.py
|   |— validation.py
|   |— report.py
|— reports/
|— charts/
|— screenshots/

```

9. Module Descriptions

File	Responsibility
app.py	Main Streamlit application. Handles the interface, file upload, workflow, KPIs, charts, insights and downloads.
modules/cleaning.py	Contains the <code>clean_data()</code> function and performs reusable cleaning operations.
modules/validation.py	Checks dataset size, duplicates, missing values and supported quality conditions.
modules/analysis.py	Calculates KPIs, numeric summaries, categorical summaries, column-level analysis and automatic insights.
modules/visualization.py	Creates numeric distributions, categorical charts, date trends and correlation visualizations.
modules/report.py	Creates an Excel workbook containing KPI, data-quality, column-analysis, numeric-summary and cleaned-data sheets.
requirements.txt	Lists Python dependencies needed to run the application.
.gitignore	Prevents files such as the virtual environment, Python cache files and local environment files from being committed.

10. Data Cleaning Process

13. Column names are stripped of surrounding whitespace and repeated spaces are converted to underscores.
14. Completely empty rows are removed.
15. Completely empty columns are removed.
16. Duplicate rows are removed.
17. Text columns are converted to string-compatible values and surrounding spaces are removed.
18. Blank strings are converted to missing values.
19. Columns whose names suggest dates (for example Date, Time, DOB, Created or Updated) are converted when valid date values are detected.
20. Object/string columns are tested for numeric compatibility; strongly numeric columns are converted to numeric data types.
21. Missing values in numeric columns are filled with the column median.

Important: automatic cleaning is intentionally general-purpose. A production system should add dataset-specific rules when business meaning is critical.

11. Data Validation

Validation provides a simple quality report before and after processing. The current validation design reports rows, columns, duplicate rows, total missing values, and additional checks for Quantity, Sales and Date when those columns exist.

For a fully generic production release, validation can be expanded to automatically detect invalid numeric ranges, impossible dates, suspicious data types, outliers, duplicate identifiers and schema changes.

12. Data Analysis and KPIs

The analysis module is designed to work without assuming a fixed business dataset. It detects numeric columns and calculates dataset-level metrics.

- Total Rows
- Total Columns
- Missing Values
- Duplicate Rows
- Numeric Total
- Numeric Average
- Numeric descriptive statistics
- Categorical unique-value and missing-value summaries
- Column-level data type, missing-value, unique-value and duplicate-value analysis

13. Automatic Visualization

The application detects column types and creates suitable visualizations automatically.

Detected Data	Visualization
Numeric columns	Histogram/distribution chart for selected numeric columns.
Categorical columns	Top-value bar charts using the most frequent categories.
Date + numeric column	Line chart showing the selected numeric measure over time.
Two or more numeric columns	Correlation matrix visualization.

14. Automatic Insights

The application generates readable observations from the processed dataset. For numeric fields it can report average, maximum and minimum values. For categorical fields it can identify the most common value. It also reports dataset size and whether missing values remain.

15. Report Generation

The Excel report is generated using Pandas and OpenPyXL. The workbook is organized into multiple sheets so that the output can be used as a professional project deliverable.

- KPI Summary — general calculated metrics.

- Data Quality — validation results.
- Column Analysis — data type, missing values and unique values.
- Numeric Summary — descriptive statistics for numeric columns.
- Cleaned Data — the final processed dataset.

The application also provides a direct CSV download of the cleaned dataset.

16. User Interface and Animations

The interface uses a dark dashboard-style design, centered project title, KPI cards, rounded controls, file upload area, progress indicators and expandable raw-data sections.

The current animation approach focuses on useful feedback rather than decorative effects. A loading spinner appears during startup, while the cleaning process uses staged progress messages such as inspection, cleaning, validation and analysis. This makes processing activity visible to the user.

17. Input Data and Testing Dataset

The project has been tested conceptually with intentionally messy structured tabular data. A large final test dataset was prepared with approximately 1,025 rows and 10 columns. It includes duplicate records, missing values, extra spaces, inconsistent capitalization, mixed date formats, invalid dates, negative quantities, missing prices/sales and missing emails.

Example columns include: Order ID, Customer Name, Date, Region, Category, Product, Quantity, Price, Sales and Email.

This type of dataset is suitable for demonstrating the application's cleaning and reporting capabilities because it contains several common real-world data-quality problems.

18. Error Handling and Limitations

Supported use cases include structured CSV and XLSX files. The project is not intended to directly process:

- Images as datasets
- PDF documents containing unstructured tables
- Audio or video files
- Complex nested JSON without preprocessing
- GIS/geospatial formats without additional libraries
- Encrypted/password-protected Excel files
- Relational databases with multiple linked tables without an added database connector
- Extremely large datasets that exceed available memory

For a polished production version, file-reading operations should be wrapped in try/except blocks so malformed CSVs, corrupted Excel files, empty files and unsupported structures show friendly error messages instead of application tracebacks.

19. How to Run the Project Locally

Windows CMD workflow:

```
cd /d "C:\Users\HP\Documents\Data-Cleaning-Reporting-Automation"
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

After the Streamlit server starts, open the local address shown in the terminal, normally <http://localhost:8501>.

20. GitHub and Streamlit Deployment

For sharing with friends, teachers or recruiters, the recommended deployment flow is:

22. Create a GitHub repository for the project.
23. Commit and push app.py, modules, requirements.txt, README.md and other required project files.
24. Sign in to Streamlit Community Cloud with GitHub.
25. Create a new app and select the GitHub repository.
26. Choose app.py as the main file.
27. Deploy the application.
28. Streamlit Community Cloud provides a public streamlit.app address that can be shared.

Do not commit the local venv folder. The .gitignore file already excludes venv/.

21. User Guide

29. Open the deployed Streamlit application.
30. Click the CSV/Excel upload area.
31. Select a structured .csv or .xlsx dataset.
32. Review the Data Quality Before Cleaning section.
33. Click 'Clean & Analyze Data'.
34. Wait for the cleaning and analysis process to complete.
35. Review Before vs After Cleaning and Cleaning Impact.
36. Review KPIs, charts and automatic insights.
37. Inspect the Cleaned Dataset.
38. Download the cleaned CSV or Excel report.

22. Testing Checklist

- ☐ Application opens successfully without a file.
- ☐ Valid CSV upload works.
- ☐ Valid XLSX upload works.
- ☐ Large messy CSV can be processed.

- ☐ Duplicate rows are detected and removed.
- ☐ Missing numeric values are handled.
- ☐ Whitespace in text fields is cleaned.
- ☐ Date fields are processed where detectable.
- ☐ Before/after metrics display correctly.
- ☐ Numeric charts display when numeric columns exist.
- ☐ Categorical charts display when categorical columns exist.
- ☐ Trend chart appears when a suitable date and numeric column exist.
- ☐ Correlation chart appears when at least two numeric columns exist.
- ☐ CSV download works.
- ☐ Excel report generation works.
- ☐ No-data and unsupported scenarios are handled gracefully.

23. Future Enhancements

- Add stronger generic validation rules.
- Allow users to choose which columns should be treated as dates or numbers.
- Add configurable missing-value strategies such as median, mean, mode or removal.
- Add outlier detection and optional treatment.
- Add duplicate-key detection based on selected columns.
- Add interactive Plotly charts and richer dashboard controls.
- Add customizable report templates.
- Add PDF report generation.
- Add database input such as MySQL/PostgreSQL.
- Add authentication for private deployments.
- Add automated unit tests.
- Add logging for debugging and auditability.
- Add larger-file processing strategies such as chunked CSV reading.
- Improve deployment configuration and automated CI/CD testing.

24. Conclusion

Data Cleaning & Reporting Automation demonstrates how Python, Pandas, Streamlit and Excel reporting can be combined to build a practical data-analytics application. The project converts a raw structured dataset into a cleaner, validated and analyzed dataset while reducing repetitive manual work.

The modular design also makes the project suitable for further development. It can be extended from a student portfolio project into a more robust data-quality and reporting platform by adding stronger validation, configurable cleaning rules, testing, richer visualizations, database support and production-grade error handling.

Project Summary

Area	Implementation
Input	CSV / Excel
Processing	Cleaning + Validation + Analysis
Visualization	Automatic charts and insights
Output	Cleaned CSV + Excel report
Interface	Streamlit web application
Version Control	Git / GitHub
Deployment	Streamlit Community Cloud